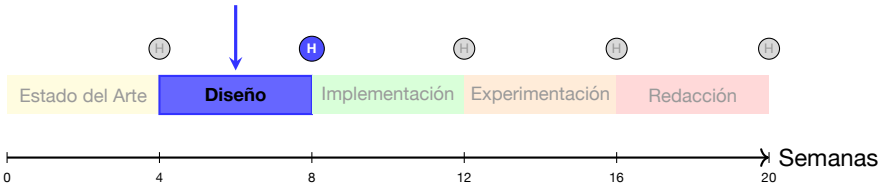
Documento hito #5 (de 6) de mes 2



Entorno Spark: Configuración y Validación

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Notebook: notebooks/00_spark_setup_template.ipynb

Como paso previo al desarrollo del algoritmo genético, se ha creado un notebook de configuración y validación del entorno Spark. El objetivo es verificar que PySpark está correctamente instalado y que la sesión puede inicializarse con los parámetros necesarios para el proyecto, antes de incorporar Spark como motor de paralelización en los experimentos principales.

El notebook cubre tres áreas: la configuración de la `SparkSession` con los parámetros de memoria y paralelismo adecuados, la ejecución de operaciones básicas con RDDs y DataFrames para comprobar el correcto funcionamiento de las APIs principales, y la inspección de la Spark Web UI para confirmar que los jobs se distribuyen y ejecutan como se espera.

---

1. Configuración del entorno

La sesión de Spark se construye mediante la API `SparkSession.builder` de **PySpark** (pyspark.sql.SparkSession), que expone un patrón fluent para encadenar opciones de configuración antes de llamar a `.getOrCreate()`:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TFM_Experimento_01") \
    .master("local[*]") \
    .config("spark.driver.memory", "4g") \
    .config("spark.executor.memory", "4g") \
    .config("spark.sql.shuffle.partitions", "4") \
    .config("spark.default.parallelism", "4") \
    .config("spark.ui.enabled", "true") \
    .config("spark.ui.port", "4040") \
    .config("spark.driver.host", "localhost") \
    .config("spark.driver.bindAddress", "127.0.0.1") \
    .config("spark.sql.execution.arrow.pyspark.enabled", "true") \
    .getOrCreate()
```

La localización de la instalación de Spark en el sistema se delega en **findspark** (findspark), que llama a `findspark.init()` para añadir el directorio de PySpark al `sys.path` antes de cualquier importación, evitando la necesidad de configurar manualmente `SPARK_HOME` o `PYTHONPATH`.

El nivel de log se ajusta a través de `spark.sparkContext.setLogLevel("WARN")` para suprimir mensajes informativos durante la ejecución del notebook y reducir el ruido en la salida.

Los parámetros resultantes de la sesión son:

Parámetro	Valor
App Name	`TFM_Experimento_01`
Master	`local[*]` (todos los cores disponibles)
Spark Version	3.5.0
Python Version	3.9
Driver Memory	4 GB
Executor Memory	4 GB
Default Parallelism	4
Shuffle Partitions	4
Arrow para Pandas	habilitado
Spark Web UI	http://localhost:4040

El modo `local[*]` instruye a Spark para que use todos los núcleos de la máquina como workers dentro del mismo proceso JVM, proporcionando paralelismo real sin necesidad de un clúster externo. Con 4 cores disponibles, Spark crea 4 particiones por defecto para distribuir el trabajo. La opción `spark.sql.execution.arrow.pyspark.enabled` activa la serialización mediante **Apache Arrow** (pyarrow) en las conversiones entre DataFrames de Spark y Pandas, reduciendo significativamente el coste de `.toPandas()`.

---

2. Ejecución de ejemplos simples con RDDs

Para validar el entorno se ejecutan cuatro tests progresivos que cubren las principales APIs de Spark:

Test 1 — RDD básico con transformación `map`

```
rdd = spark.sparkContext.parallelize(range(1_000_000))
resultado = rdd.map(lambda x: x**2).sum()
# → Suma de cuadrados (0 a 999999): 333,332,833,333,500,000
```

Se crea un RDD de 1 millón de elementos, se aplica una transformación `map` (operación lazy) y se materializa con la acción `sum`. Spark divide el rango en particiones distribuidas entre los workers locales.

Test 2 — DataFrame con schema explícito

```
datos = [("Alice", 25, "Engineering"), ("Bob", 30, "Finance"), ...]
df_spark = spark.createDataFrame(datos, ["Nombre", "Edad", "Departamento"])
df_spark.show()
```

Valida la creación de DataFrames estructurados y la inferencia de tipos (`string`, `long`).

Test 3 — Spark SQL sobre vista temporal

```
df_spark.createOrReplaceTempView("empleados")
resultado_sql = spark.sql("""
    SELECT Departamento, COUNT(*) as cantidad, AVG(Edad) as edad_promedio
    FROM empleados GROUP BY Departamento ORDER BY cantidad DESC
""")
```

Confirma que el motor SQL de Spark funciona correctamente sobre DataFrames registrados como vistas.

Test 4 — Conversión a Pandas y visualización

Se ejecuta `.toPandas()` (acelerado por Arrow) para recolectar los resultados en el driver y generar visualizaciones con **matplotlib** y **seaborn**. Este paso valida la interoperabilidad entre el DataFrame de PySpark y el ecosistema científico de Python: los datos procesados en el clúster se transfieren al proceso driver como un `pandas.DataFrame` estándar, disponible para cualquier librería de análisis o visualización.

---

3. Spark Web UI

Jobs completados

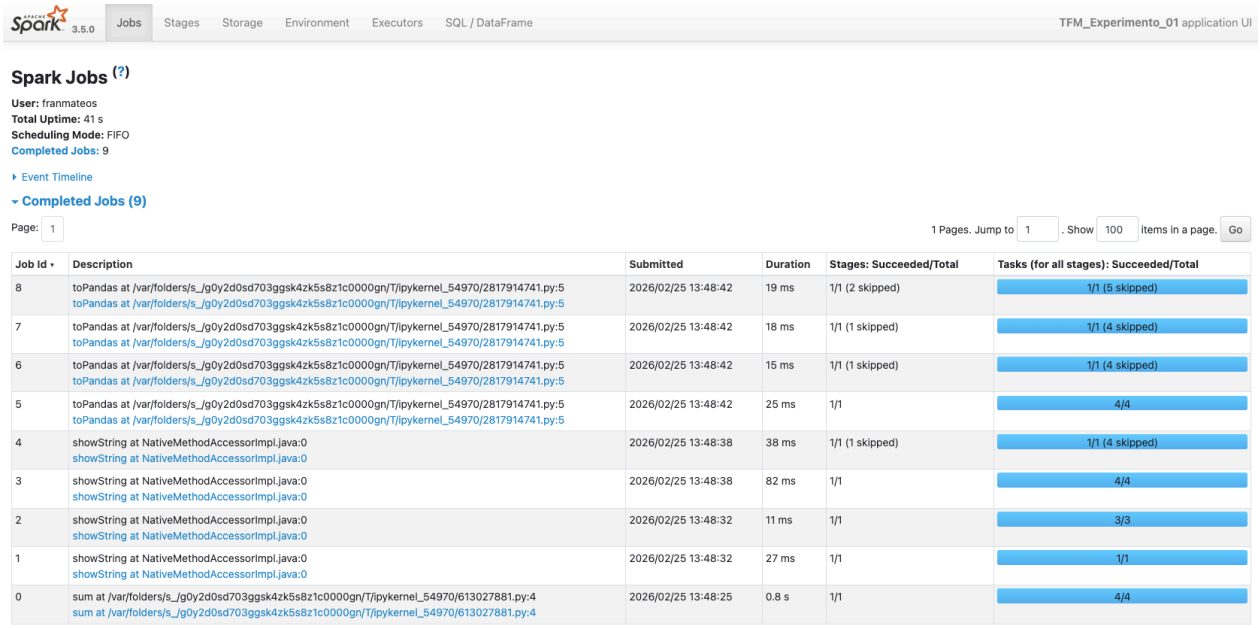


Figure 1: Spark Jobs

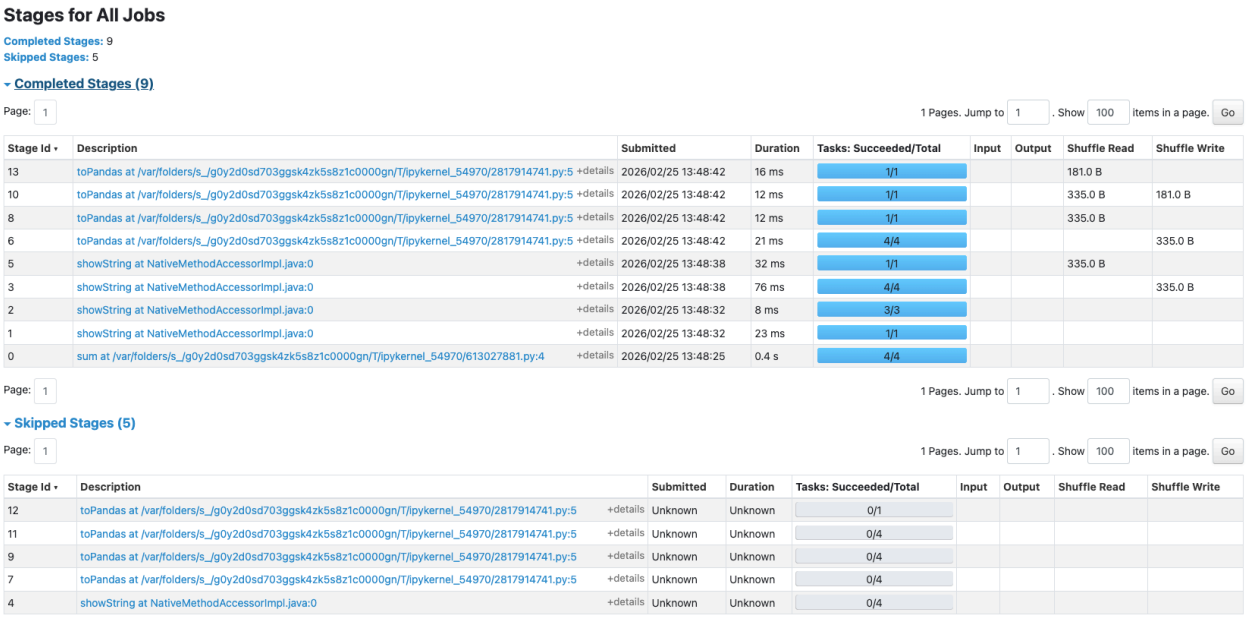
La vista **Jobs** muestra los **9 jobs completados** durante la sesión de validación. Cada test del notebook genera uno o varios jobs:

- Los jobs asociados a `parallelize` + `map` + `sum` (Test 1) producen stages con múltiples tasks ejecutadas en paralelo.
- Los jobs de DataFrame y SQL (Tests 2–4) generan stages de menor duración al operar sobre datos pequeños.
- La columna **Stages** confirma la correcta división en stages (entre 1 y 12 stages por job según la complejidad).
- La columna **Tasks** refleja el paralelismo efectivo: los jobs sobre el RDD de 1M elementos distribuyen las tareas entre los 4 workers disponibles.

Stages completados y skipped

La vista **Stages** desglosa la ejecución interna de cada job:

- **9 stages completados** con métricas de duración, input/output y shuffle read/write.
- **5 stages skipped** (marcados en gris): Spark detectó que los datos ya estaban cacheados o que el resultado podía reutilizarse, evitando recomputación innecesaria — indicador de que la optimización del planificador (Catalyst) está activa.



Stages for All Jobs

Completed Stages: 9
Skipped Stages: 5

Completed Stages (9)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
13	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	2026/02/25 13:48:42	16 ms	1/1			181.0 B	
10	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	2026/02/25 13:48:42	12 ms	1/1			335.0 B	181.0 B
8	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	2026/02/25 13:48:42	12 ms	1/1			335.0 B	
6	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	2026/02/25 13:48:42	21 ms	4/4				335.0 B
5	showString at NativeMethodAccessorImpl.java:0 +details	2026/02/25 13:48:38	32 ms	1/1			335.0 B	
3	showString at NativeMethodAccessorImpl.java:0 +details	2026/02/25 13:48:38	76 ms	4/4				335.0 B
2	showString at NativeMethodAccessorImpl.java:0 +details	2026/02/25 13:48:32	8 ms	3/3				
1	showString at NativeMethodAccessorImpl.java:0 +details	2026/02/25 13:48:32	23 ms	1/1				
0	sum at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/613027881.py:4 +details	2026/02/25 13:48:25	0.4 s	4/4				

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Skipped Stages (5)

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
12	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	Unknown	Unknown	0/1				
11	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	Unknown	Unknown	0/4				
9	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	Unknown	Unknown	0/4				
7	toPandas at /var/folders/s_/g0y2d0sd703ggsk4zk5s8z1c0000gn/T/ipykernel_54970/2817914741.py:5 +details	Unknown	Unknown	0/4				
4	showString at NativeMethodAccessorImpl.java:0 +details	Unknown	Unknown	0/4				

Figure 2: Spark Stages

- Los stages del RDD grande muestran **shuffle write** de ~332 B, resultado del `sum` agregado entre particiones.
- Las duraciones oscilan entre **<1 ms** (stages triviales) y **~55 ms** (stages con transformaciones sobre 1M elementos), tiempos coherentes con una ejecución local en modo `local[*]`.

---

4. Conclusión

La sesión de Spark queda validada con los siguientes criterios:

- **Spark 3.5.0** se inicializa correctamente en modo `local[*]` con 4 workers efectivos.
- Los RDDs se crean, transforman y materializan sin errores, con distribución real del trabajo entre particiones.
- DataFrames, Spark SQL y la conversión a Pandas funcionan con coherencia de tipos y resultados correctos.
- La Web UI refleja la ejecución completa: 9 jobs, stages con tasks paralelas, optimización por skip de stages ya cacheados y métricas de shuffle.

El entorno está listo para soportar los experimentos de evaluación paralela del Algoritmo Genético definidos en fases posteriores del TFM.